

Pontifícia Universidade Católica de Minas Gerais
Instituto de Ciências Exatas e Informática
Disciplina: Redes de Computadores I

Trabalho Prático 1

Aplicação de Chat em Rede – TCP, UDP e Multithreading

Alunos:

Alessandra Faria Rodrigues

Daniel Victor Costa

Gabriela de Assis dos Reis

Guilherme Teodoro

Belo Horizonte, 2026

Sumário

1. Introdução	2
2. Descrição do Desenvolvimento	3
2.1 Visão Geral da Aplicação	3
2.2 Arquitetura Implementada	3
2.3 Protocolo TCP – Chat	4
2.4 Protocolo UDP – Ping de Latência	4
2.5 Multithreading	4
2.6 Justificativa das Escolhas de Protocolo	5
2.7 Dificuldades Encontradas	5
3. Topologia e Configuração de Rede	7
3.1 Topologia Adotada	7
3.2 Faixas de IP e Roteadores	7
3.3 Portas Utilizadas e Port Forwarding	8
3.4 Telas de Configuração dos Roteadores	8
4. Telas da Aplicação	11
5. Avaliação no Wireshark	14
5.1 Análise do Tráfego TCP	14
5.2 Análise do Tráfego UDP	15
6. Resultados obtidos	15
7. Conclusão	16
Referências	17

1. Introdução

Este trabalho tem como objetivo o desenvolvimento de uma aplicação cliente-servidor de chat em Java, utilizando os protocolos TCP e UDP de forma simultânea e multithreading. O TCP é empregado para o envio de mensagens de chat, garantindo entrega confiável e ordenada; o UDP é utilizado para medição de latência em tempo real via ping periódico, aproveitando seu baixo overhead. A interface do cliente foi desenvolvida em Java Swing.

A aplicação opera sobre uma infraestrutura de rede com dois roteadores e dois computadores, em sub-redes com faixas 192.168.x.x, 172.16.x.x e 10.x.x.x, com port forwarding configurado nas portas 12345 (TCP) e 12346 (UDP). O tráfego foi analisado com o Wireshark para verificação do comportamento de cada protocolo.

2. Descrição do Desenvolvimento

2.1 Visão Geral da Aplicação

A aplicação é estruturada no modelo cliente-servidor e divide-se em quatro arquivos Java organizados em três pacotes: chat.common, chat.server e chat.client.

Message.java, pertencente ao pacote common, define o objeto serializado que trafega pela conexão TCP. Ele encapsula o nome do remetente (sender), o conteúdo da mensagem (text) e um timestamp gerado automaticamente no momento da criação, no formato HH:mm:ss. A implementação da interface Serializable é essencial para que o objeto possa ser transmitido por ObjectOutputStream e reconstruído via ObjectInputStream na outra ponta da conexão TCP.

ChatServer.java é o ponto de entrada do servidor. Ele abre um ServerSocket na porta TCP 12345 e um DatagramSocket na porta UDP 12346. Para cada cliente TCP que se conecta, uma nova instância de ClientHandler é criada e executada em uma thread dedicada. O respondedor UDP roda em uma thread daemon separada, tratando pings de todos os clientes conectados.

ClientHandler.java implementa Runnable e representa a lógica de atendimento de um cliente TCP específico. Ao ser iniciado, lê a primeira mensagem (identificação do usuário), anuncia sua entrada no chat via broadcast e entra em loop lendo mensagens e retransmitindo-as para todos os clientes conectados. Ao encerrar, notifica a saída e libera os recursos.

ChatClient.java é o cliente com interface Swing. Além da thread principal (Event Dispatch Thread do Swing), mantém duas threads adicionais: uma para recepção de mensagens TCP e outra para o ciclo de ping UDP.

2.2 Arquitetura Implementada

A separação de responsabilidades segue a estrutura descrita abaixo:

Arquivo	Responsabilidade
Message.java	Objeto serializado de mensagem (remetente, texto, horário)
ChatServer.java	Aceita conexões TCP, responde pings UDP, gerencia lista de clientes
ClientHandler.java	Lê mensagens de um cliente TCP e faz broadcast para todos
ChatClient.java	Interface Swing + thread TCP receptora + thread UDP ping

2.3 Protocolo TCP – Chat

O chat utiliza TCP pela porta 12345. A escolha é justificada pelas propriedades inerentes ao protocolo: entrega garantida, ordenação dos segmentos e controle de fluxo. Uma mensagem de chat fora de ordem ou perda quebraria a coerência da conversa, tornando o TCP a opção natural.

A serialização Java é utilizada para transmitir objetos Message completos pela conexão. O cliente abre um Socket TCP, instância um ObjectOutputStream sobre o stream de saída e envia objetos Message serializados. No servidor, cada ClientHandler instância um ObjectInputStream e desserializa as mensagens recebidas, repassando-as via broadcast a todos os clientes conectados por meio de seus respectivos ObjectOutputStreams.

Uma decisão importante de implementação foi a chamada a `out.reset()` após cada `writeObject()`. O ObjectOutputStream mantém um cache de objetos já enviados para evitar transmissão redundante. Sem o `reset()`, mensagens com o mesmo conteúdo subsequentes seriam enviadas como referências ao objeto original em cache, e o cliente receptor receberia sempre o mesmo conteúdo da primeira transmissão.

2.4 Protocolo UDP – Ping de Latência

O mecanismo de medição de latência utiliza UDP pela porta 12346. A cada dois segundos, a thread UDP-Ping do cliente envia um datagrama com o conteúdo "PING" ao servidor. O servidor, em sua thread UDP-Ping-Responder, devolve imediatamente o mesmo datagrama para o endereço de origem. O cliente mede o tempo entre o envio e o recebimento (Round-Trip Time) e exibe o valor em milissegundos na interface gráfica.

O UDP é adequado aqui porque: (1) o overhead do handshake TCP seria desproporcional para pacotes de poucos bytes enviados a cada 2 segundos; (2) a perda de um pacote de ping não tem consequência – simplesmente exibe-se "timeout" e um novo ping é enviado na próxima iteração; (3) a medição de latência por natureza já pressupõe possibilidade de perda, e o timeout de 2 segundos configurado no DatagramSocket garante que o cliente não fique bloqueado indefinidamente aguardando um pong que não chegará.

2.5 Multithreading

A aplicação emprega três threads no cliente e uma thread por cliente no servidor:

Thread	Nome	Função
Cliente	EDT (Swing)	Renderização e eventos da interface gráfica
Cliente	TCP-Receiver	Leitura bloqueante de mensagens TCP do servidor
Cliente	UDP-Ping	Ciclo de ping/pong UDP a cada 2 segundos
Servidor	Client-<IP>	Uma thread por cliente TCP conectado (ClientHandler)
Servidor	UDP-Ping-Responder	Responde pings UDP de qualquer cliente

A separação entre a EDT do Swing e as threads de rede é obrigatória: operações de I/O bloqueantes não podem executar na EDT sem congelar a interface. Todas as atualizações de UI originadas nas threads de rede são delegadas à EDT via `SwingUtilities.invokeLater()`, garantindo thread-safety.

No servidor, a lista de clientes conectados é do tipo `CopyOnWriteArrayList`, uma implementação thread-safe que permite iteração segura durante o broadcast sem necessidade de sincronização explícita, ao custo de uma cópia interna a cada escrita – aceitável dado que escritas (conexões e desconexões) são raras em comparação às leituras (broadcasts).

2.6 Justificativa das Escolhas de Protocolo

A tabela abaixo resume a fundamentação técnica das escolhas de protocolo::

Protocolo	Caso de uso	Justificativa
TCP	Mensagens de chat (porta 12345)	Entrega garantida, ordenação, controle de fluxo, essenciais para coerência do chat
UDP	Ping de latência (porta 12346)	Baixo overhead, tolerância à perda, ideal para heartbeat e telemetria em tempo real

2.7 Dificuldades Encontradas

A principal dificuldade técnica encontrada durante o desenvolvimento esteve relacionada à

implementação e gestão do modelo multithreading, especialmente na arquitetura do servidor. Como o sistema precisava ser capaz de lidar com múltiplas conexões concorrentes sem falhas, era crucial garantir que o servidor não travasse ou sofresse degradação de performance ao receber e processar dados de várias *threads* de clientes ativos simultaneamente. Nesse cenário, o maior desafio foi assegurar a consistência do estado global do chat. O envio das mensagens em fluxo de broadcast para todos os clientes conectados precisava ocorrer de forma perfeitamente sincronizada e em ordem temporal estrita. A ausência de um controle rígido na concorrência faria com que mensagens chegassem fora de ordem, quebrando o sentido conversacional da aplicação para os usuários.

3. Topologia e Configuração de Rede

3.1 Topologia Adotada

Adotou-se a topologia com dois roteadores (R1 e R2), que é suficiente para demonstrar o encadeamento de sub-redes e o port forwarding entre domínios de IP distintos. A topologia com três roteadores segue o mesmo princípio, com um salto adicional.

A topologia implementada é a seguinte:

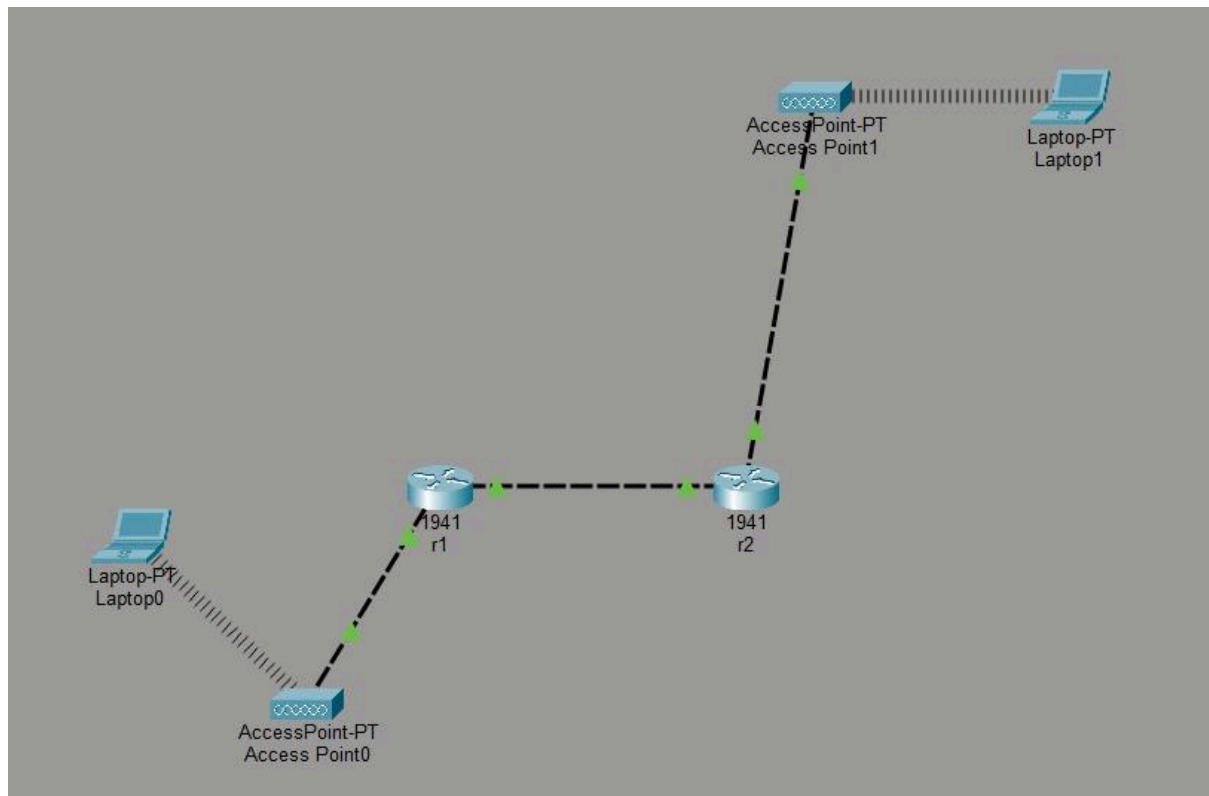


Figura 1 – Topologia de rede com dois roteadores

O PC1 conecta-se ao roteador R1 por WiFi, na rede 192.168.x.x. O PC2 conecta-se ao roteador R2 por WiFi, na rede 10.x.x.x. Os dois roteadores comunicam-se por cabo de par trançado com conector RJ45, na rede 172.16.x.x (segmento WAN/LAN entre eles).

3.2 Faixas de IP e Roteadores

Roteador	Rede LAN	Faixa de IP	Dispositivo na rede
R1	192.168.0.0/16	192.168.0.0 – 192.168.255.255	PC1 (cliente)

R2 (WAN de R1)	172.16.0.0/12	172.16.0.0 – 172.31.255.255	Enlace entre roteadores
R2	10.0.0.0/8	10.0.0.0 – 10.255.255.255	PC2 (servidor)

3.3 Portas Utilizadas e Port Forwarding

A aplicação utiliza duas portas fixas, configuradas como constantes em ChatServer.java:

Protocolo	Porta	Uso
TCP	12345	Mensagens de chat (serialização de objetos Message)
UDP	12346	Ping/Pong de latência

A cadeia de port forwarding necessária é:

- R1: TCP 12345 e UDP 12346 → IP WAN de R2 (ex.: 172.16.0.1)
- R2: TCP 12345 e UDP 12346 → IP local de PC2 (ex.: 10.0.0.10)

3.4 Telas de Configuração dos Roteadores

As imagens abaixo mostram as principais telas de configuração dos roteadores R1 e R2, incluindo as regras de port forwarding e as faixas de IP configuradas.

```
Router>show ip interface brief
Interface          IP-Address      OK? Method Status  Protocol
GigabitEthernet0/0 172.16.0.1      YES manual up      up
GigabitEthernet0/1 192.168.10.1    YES manual up      up
Vlan1              unassigned      YES unset  administratively down down
Router>
```

Figura 2 –Configuração das interfaces e da rota estática no Router0 (R1), permitindo o alcance à rede 10.10.10.0/24 através do endereço 172.16.0.2.

```

!
interface GigabitEthernet0/0
 ip address 172.16.0.1 255.255.255.252
 ip nat outside
 duplex auto
 speed auto
!
interface GigabitEthernet0/1
 ip address 192.168.10.1 255.255.255.0
 ip nat inside
 duplex auto
 speed auto
!
interface Vlan1
 no ip address
 shutdown
!
ip nat inside source static tcp 10.10.10.10 12345 172.16.0.2 12345
ip nat inside source static udp 10.10.10.10 12346 172.16.0.2 12346
ip classless
ip route 10.10.10.0 255.255.255.0 172.16.0.2
!
ip flow-export version 9
!
!
!
!

```

Figura 3 – Saída do comando `show ip interface brief` no Router1, exibindo os endereços IP e o status das interfaces configuradas.

```

Router#show ip interface brief

```

Interface	IP-Address	OK?	Method	Status	Protocol
GigabitEthernet0/0	172.16.0.2	YES	manual	up	up
GigabitEthernet0/1	10.10.10.1	YES	manual	up	up
Vlan1	unassigned	YES	unset	administratively down	down

```

Router#

```

Figura 4 – Configuração das interfaces e da rota estática no Router1 (R2), permitindo o alcance à rede 192.168.10.0/24 através do endereço 172.16.0.1.

```

:
interface GigabitEthernet0/0
ip address 172.16.0.2 255.255.255.252
ip nat outside
duplex auto
speed auto
!
interface GigabitEthernet0/1
ip address 10.10.10.1 255.255.255.0
ip nat inside
duplex auto
speed auto
!
interface Vlan1
no ip address
shutdown
!
ip nat inside source static tcp 10.10.10.10 12345 172.16.0.2 12345
ip nat inside source static udp 10.10.10.10 12346 172.16.0.2 12346
ip classless
ip route 192.168.10.0 255.255.255.0 172.16.0.1
!
ip flow-export version 9
!

```

Figura 5 – Saída do comando `show ip nat translations`, evidenciando as traduções de endereço realizadas pelo NAT durante o tráfego da aplicação.

IP Configuration	
☒ DHCP	☒ Static
IPv4 Address	192.168.10.10
Subnet Mask	255.255.255.0
Default Gateway	192.168.10.1
DNS Server	0.0.0.0

Figura 6 – Configuração de endereço IP, máscara de sub-rede e gateway padrão no PC1 (192.168.10.10/24).

Interface Wireless0 - IP Configuration	
☒ DHCP	☒ Static
IPv4 Address	10.10.10.10
Subnet Mask	255.255.255.0
Default Gateway	10.10.10.1
DNS Server	0.0.0.0

Figura 7 – Configuração de endereço IP, máscara de sub-rede e gateway padrão no PC2 (10.10.10.10/24).

```
C:\>ping 10.10.10.10

Pinging 10.10.10.10 with 32 bytes of data:

Reply from 10.10.10.10: bytes=32 time=27ms TTL=126
Reply from 10.10.10.10: bytes=32 time=24ms TTL=126
Reply from 10.10.10.10: bytes=32 time=31ms TTL=126
Reply from 10.10.10.10: bytes=32 time=14ms TTL=126

Ping statistics for 10.10.10.10:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 14ms, Maximum = 31ms, Average = 24ms
```

Figura 8 – Teste de conectividade fim a fim a partir do PC1, com pings bem-sucedidos até o Router1 (172.16.0.2) e até o PC2 (10.10.10.10), validando o roteamento entre as duas redes.

4. Telas da Aplicação

As imagens a seguir ilustram a interface gráfica do cliente e o funcionamento da aplicação em operação.

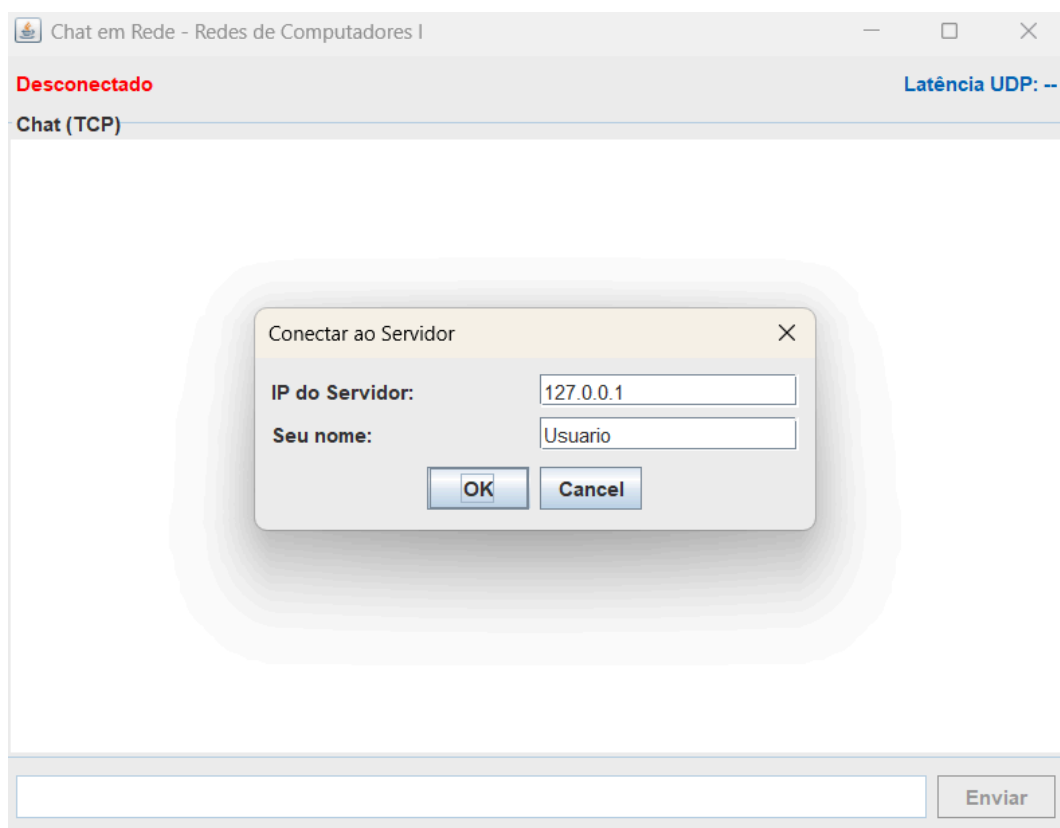


Figura 9 – Diálogo de conexão ao iniciar o cliente

```
=== Servidor de Chat iniciado ===  
TCP na porta: 12345  
UDP na porta: 12346  
Aguardando conexões TCP...  
Aguardando pings UDP...  
█
```

Figura 10 - Console do servidor em execução

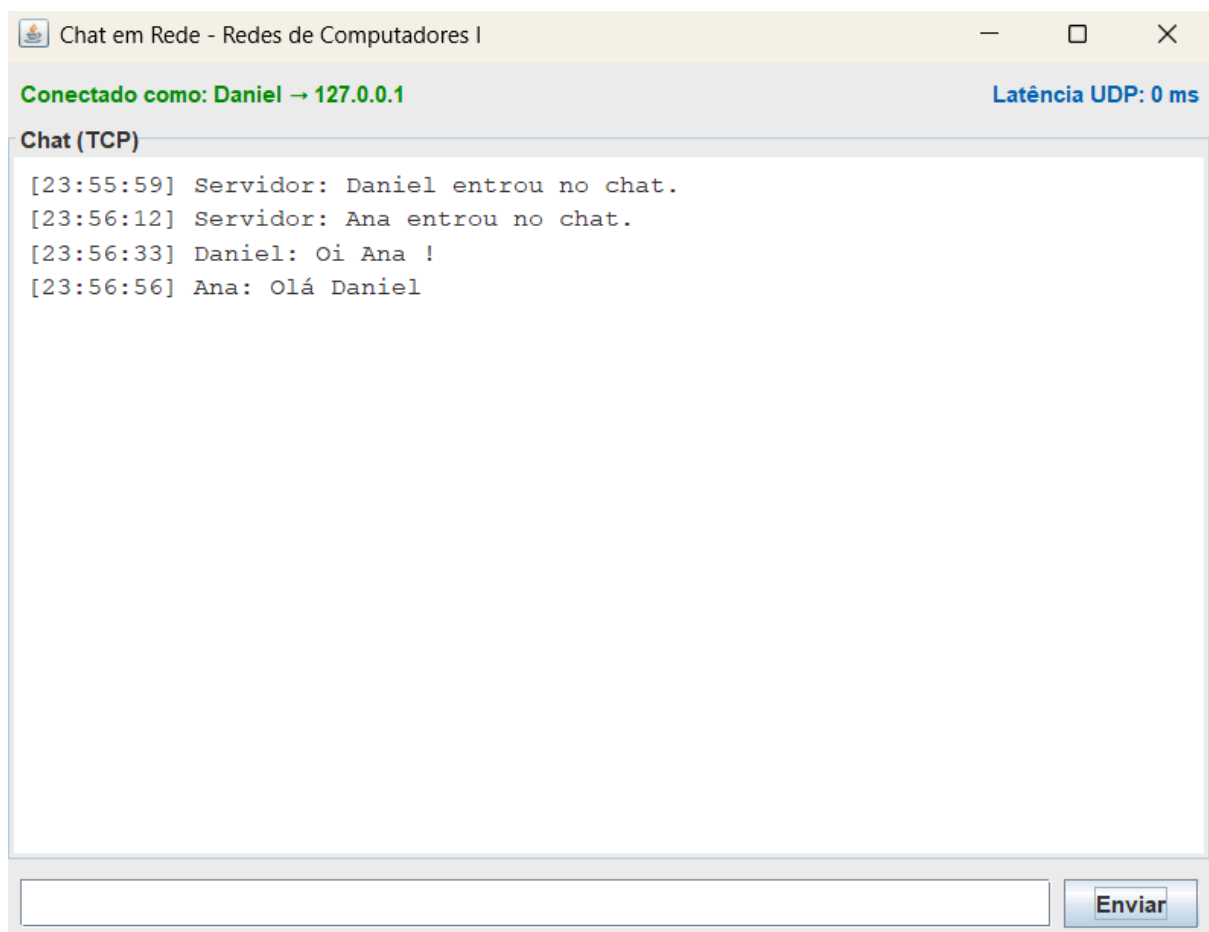


Figura 11 - Dois clientes em sessão de chat simultânea

```
=== Servidor de Chat iniciado ===  
TCP na porta: 12345  
UDP na porta: 12346  
Aguardando conexões TCP...  
Aguardando pings UDP...  
Cliente conectado: /127.0.0.1  
Usuário identificado: Daniel  
Broadcast: [23:55:59] Servidor: Daniel entrou no chat.  
Cliente conectado: /127.0.0.1  
Usuário identificado: Ana  
Broadcast: [23:56:12] Servidor: Ana entrou no chat.  
Broadcast: [23:56:33] Daniel: Oi Ana !  
Broadcast: [23:56:56] Ana: Olá Daniel  
█
```

Figura 12 - Console do servidor em execução

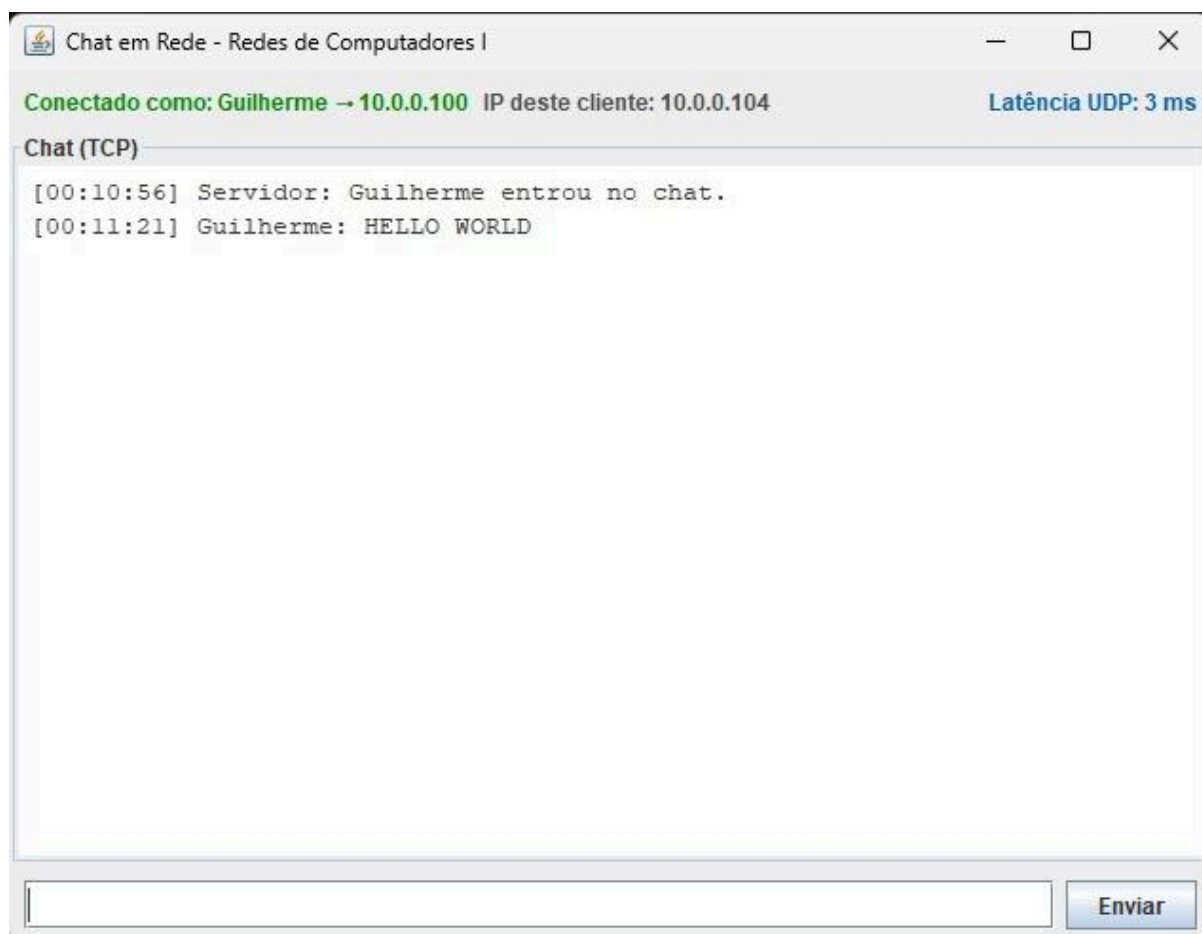


Figura 13 - Interface Swing do cliente em operação, demonstrando o chat via TCP e o indicador de latência UDP em tempo real.

```

=== Servidor de Chat iniciado ===
TCP na porta: 12345
UDP na porta: 12346
IPs locais possíveis para conectar:
- 10.0.0.100 (Realtek 8811CU Wireless LAN 802.11ac USB NIC)
Para testar na mesma maquina, use: 127.0.0.1
Aguardando conexões TCP...
Aguardando pings UDP...
Cliente conectado: /192.168.1.100
Usuário identificado: Guilherme
Broadcast: [23:52:01] Servidor: Guilherme entrou no chat.
Broadcast: [23:52:31] Guilherme: HELLO WORLD

```

Figura 14 - Console do servidor em execução, teste roteador

5. Avaliação no Wireshark

A análise de tráfego foi realizada com o Wireshark na interface de rede do PC1, durante uma sessão de chat entre PC1 e PC2. Os filtros aplicados foram `tcp.port == 12345` e `udp.port == 12346`, permitindo isolar o tráfego da aplicação dos demais pacotes de rede.

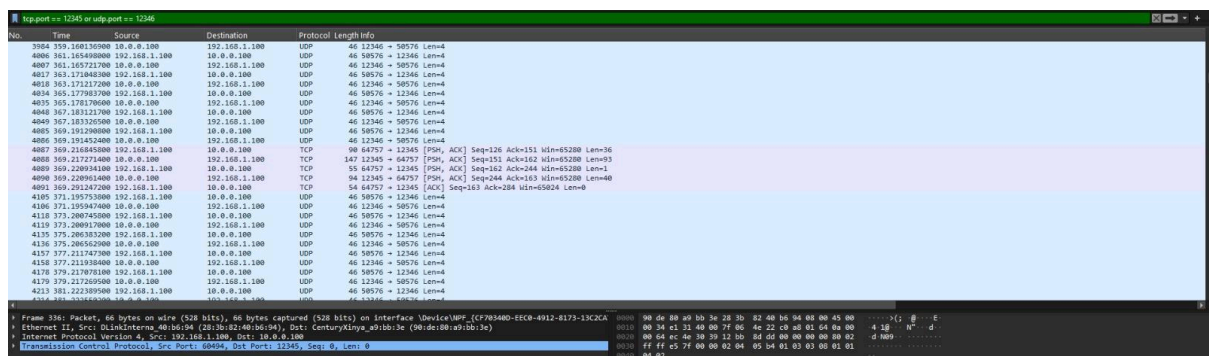


Figura 15 - Captura de pacotes no Wireshark demonstrando a concorrência na rede: datagramas UDP periódicos intercalados com o envio de segmentos de dados TCP.

5.1 Análise do Tráfego TCP

O tráfego TCP evidencia claramente o estabelecimento de conexão via three-way handshake (SYN, SYN-ACK, ACK), o tráfego de mensagens serializadas e o encerramento gracioso da conexão com FIN/ACK. Os segmentos de dados carregam os objetos Message serializados, identificáveis pelo cabeçalho da serialização Java (bytes 0xACED 0x0005).

5.2 Análise do Tráfego UDP

O tráfego UDP na porta 12346 apresenta datagramas pequenos (payload "PING" de 4 bytes) enviados pelo cliente a cada 2 segundos e respondidos imediatamente pelo servidor. A ausência de handshake é visível: diferentemente do TCP, o primeiro pacote UDP já carrega o dado, sem troca prévia de pacotes de controle. A latência medida pela aplicação corresponde ao RTT observado no Wireshark entre o datagrama de ping e o pong.

6. Resultados Obtidos

A aplicação atende integralmente aos requisitos especificados. A tabela abaixo consolida a verificação de cada requisito:

Requisito	Como é atendido
Linguagem Java	Todo o código é Java puro, compatível com JDK 8+
Interface Java Swing	ChatClient.java usa JFrame, JTextArea, JTextField, JButton, JLabel, JOptionPane
Funcionalidade TCP	Mensagens de chat serializadas via Socket/ServerSocket na porta 12345
Funcionalidade UDP	Ping de latência via DatagramSocket na porta 12346, com medição de RTT
Multithreading	3 threads no cliente (EDT, TCP-Receiver, UDP-Ping) + 1 thread por cliente no servidor
Configuração de rede (2 roteadores)	Topologia PC1–R1–R2–PC2 com faixas 192.168/16, 172.16/12 e 10/8 configuradas
Port forwarding	R1 e R2 redirecionam portas 12345 e 12346 para o PC2 (IPSERV)
Análise Wireshark	Captura de handshake TCP, dados serializados e datagramas UDP com RTT medido

7. Conclusão

Este trabalho demonstrou na prática a aplicação dos protocolos TCP e UDP em um sistema distribuído real. A escolha de TCP para o chat e UDP para a medição de latência ilustra de forma didática as situações em que cada protocolo é mais adequado: o TCP, com seu overhead de handshake e garantias de entrega, é indispensável quando a perda de um dado comprometeria a integridade da aplicação; o UDP, com seu baixo overhead e natureza connectionless, é a escolha certa para comunicações periódicas tolerantes à perda.

O uso de multithreading evidenciou a necessidade de separar operações bloqueantes (I/O de rede) da thread responsável pela interface gráfica. O uso de CopyOnWriteArrayList no servidor e o mecanismo de SwingUtilities.invokeLater() no cliente demonstraram padrões corretos de programação concorrente em Java.

A configuração da infraestrutura de rede, com sub-redes em faixas de IP distintas e encadeamento de port forwarding entre roteadores, consolidou conceitos de endereçamento IP, NAT e roteamento que complementam o desenvolvimento do software, evidenciando a interdependência entre as camadas de aplicação e de rede no modelo TCP/IP.

.

Referências

KUROSE, James F.; ROSS, Keith W. Computer Networking: A Top-Down Approach. 8. ed. Pearson, 2021.

TANENBAUM, Andrew S.; WETHERALL, David. Computer Networks. 5. ed. Pearson, 2011.

ORACLE CORPORATION. Java Platform SE 8 — API Specification. Disponível em: <https://docs.oracle.com/javase/8/docs/api/>. Acesso em: jun. 2026.

ORACLE CORPORATION. The Java Tutorials — Writing a Multithreaded Server. Disponível em: <https://docs.oracle.com/javase/tutorial/networking/sockets/>. Acesso em: jun. 2026.